Final Project Report

Yiyang Huang
huang.yiyang@northeastern.edu

Course: Foundations of Artificial Intelligence (CS 5100)
GitHub Repository: `https://github.com/averatec0773/cs5100-final-project`

1 Overview

In Phase 2, this project implemented Feature Selection and the Neural Network model, building on the work completed in Phase 1. Feature selection was performed by filtering based on feature variance and Pearson correlation coefficients, reducing the number of features from 56 to 15. Additionally, for the neural network model, a Multilayer Perceptron (MLP) is trained and tested on both the full dataset and the feature-filtered dataset. Finally, the results were compared with those of Gradient Boosting (GB) and Random Forest (RF) under the same settings. The experimental results show that feature selection improves the performance of GB and MLP, but has a negative impact on RF. Overall, the MLP model with feature selection achieved the best performance.

2 Feature Selection

This section explains my approach to feature selection. After removing features `'G1'`, `'G2'`, and `'G3'` from the raw data and performing data preprocessing, the dimensionality was increased from 30 to 56. Following feature selection, the dimensionality was reduced to 15. The overall method is based on the common filtering-based feature selection strategy [1], which combines feature variance analysis and Pearson correlation coefficient ranking to filter features.

2.1 Feature Variance Analysis

Feature variance is used to measure the degree of variation of a feature. Features with low variance typically contain less information and make a limited contribution to model predictions, thereby affecting model performance [2].

Figure 1 shows the variance distribution of each feature after preprocessing. It can be observed that most features have moderate variance, with only a small number showing low variance. Initially, I filtered out approximately 20 features with variance less than

0.1. However, the subsequent filtering based on Pearson correlation coefficients removed significantly more than 20 features. Therefore, I did not perform feature selection based on feature variance here, but retained the results for the analysis of the data features.

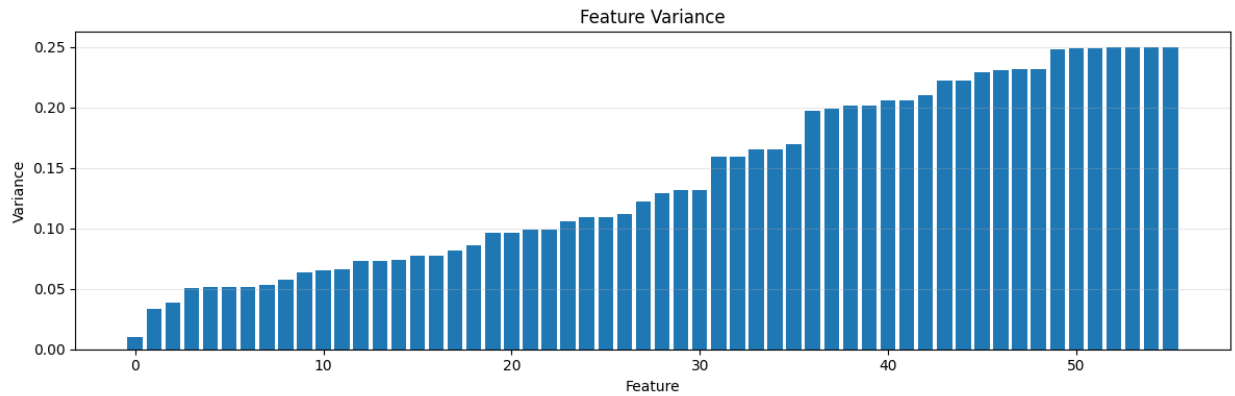

Figure 1: Feature variance on the training set.

2.2 Pearson Correlation

The Pearson correlation coefficient is used to measure the linear relationship between a feature and the target variable. A larger value indicates a stronger association between the feature and the target prediction [3][4].

For illustrative purposes, I have plotted the top 20 features with the highest Pearson correlation coefficients. Figure 2 shows the features with the strongest correlation to the target variable `'at_risk'`. As can be seen from the figure, features such as `'failures'`, `'age'`, and `'goout'` show high correlation. It is worth noting that the Pearson correlation coefficient for `'failures'` is very high, while the values for other features drop significantly. This may indicate a bias in the feature information used to predict `'at_risk'`, so I believe feature selection is necessary.

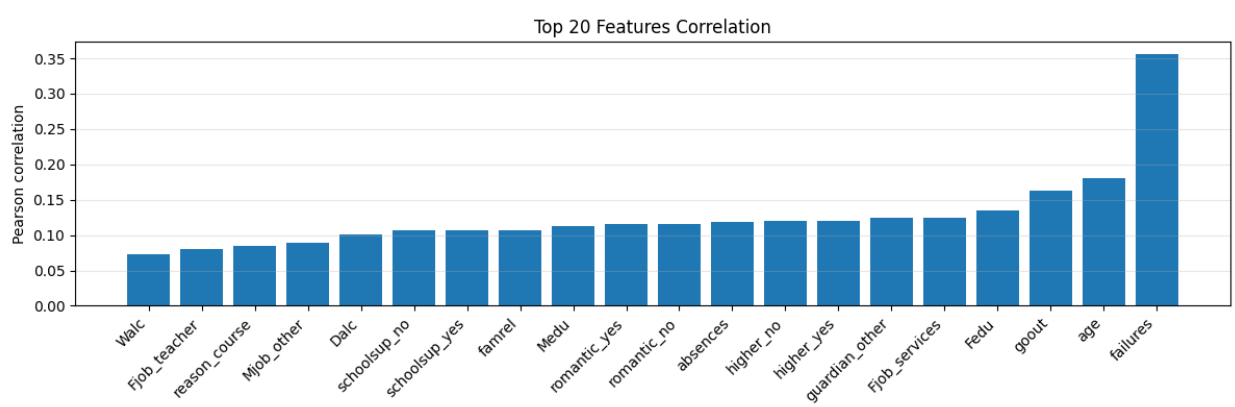

Figure 2: Pearson Correlation of Top 20 Features with the target variable (at_risk).

2.3 Selecting the Best Value of k

Since the Gradient Boosting (GB) was used as the baseline model in Phase 1, GB is also used in this phase to evaluate model performance under different numbers of features and to determine the optimal number of features, `'best_k'`. The specific method is as follows: for each candidate k value in the range $[5, 30]$, the top k features by correlation are selected, and a GB model is trained and tested, while the corresponding F1 score and ROC-AUC are calculated as evaluation metrics.

Table 1 and Figure 3 illustrate the model performance for different k values. As the number of features increases, model performance improves significantly in the early stages but subsequently stabilizes and even declines slightly.

Based on the experimental results, I finally selected $k = 15$ as the optimal number of features. This value achieves a good balance between model performance and the number of features. Another suitable value is $k = 25$, but after training the neural network with a larger k value, I found that while there was an improvement in model performance, it was not significant compared to $k = 15$. Therefore, I ultimately chose $k = 15$.

k	F1	ROC-AUC
5	0.231	0.534
10	0.500	0.587
15	0.517	0.676
20	0.509	0.645
25	0.491	0.693
30	0.444	0.687

Table 1: Model performance for different values of k using Gradient Boosting.

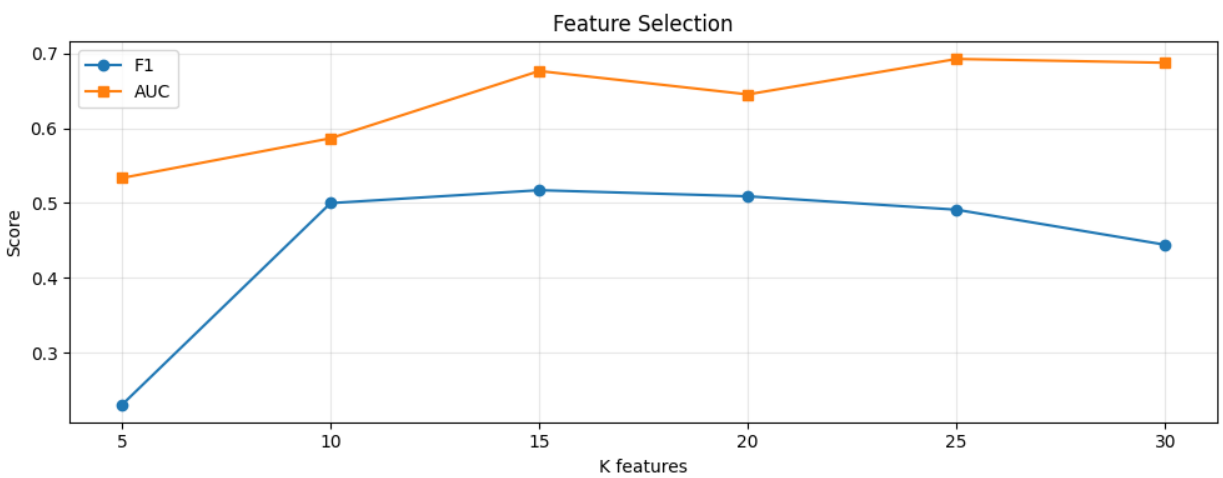

Figure 3: Model performance for different values of k using Gradient Boosting..

3 Neural Network

This section introduces the neural network model I implemented. Since this task is a binary classification problem and the dataset itself has a relatively simple structure, I did not use more complex models such as Recurrent Neural Network or Convolutional Neural Network. Instead, I chose the Multilayer Perceptron (MLP) model, which is well-suited for the current data.

3.1 Model Overview

The MLP model I used employs a fully connected architecture. I tested both single-hidden-layer and two-hidden-layer architectures. Based on the results of my experiments, the two-hidden-layer model outperformed the single-hidden-layer model. Therefore, I ultimately chose the two-hidden-layer MLP. At the same time, based on the overfitting that was later observed, I added `dropout=0.3` to both layers. The model's architecture is as follows:

```
# Layer 1
Linear(input_dim, 64)
BatchNorm1d(64)
ReLU
Dropout(0.3)
Linear(64, 32)

# Layer 2
BatchNorm1d(32)
ReLU
Dropout(0.3)
Linear(32, 1)
```

3.2 Training Setup

The model training settings use a common setting: the Adam optimizer and the Binary Cross Entropy loss function, which is frequently used for binary classification problems[5]. During training, I observed that the model began to overfit after a significant number of epochs. Therefore, I also implemented an early stopping mechanism.

3.3 Training Results

Figure 4 shows the training results of the MLP with different numbers of features. It can be observed from the figure that training for both models stopped around 25 epochs. When using all features, the validation loss showed an upward trend in the later stages of training, indicating that the model had overfit. However, when training with the feature selected dataset, the validation loss remained more stable, and the overfitting issue was reduced. Based on my understanding, feature selection might reduced the noise introduced by the features and hence improve the performance.

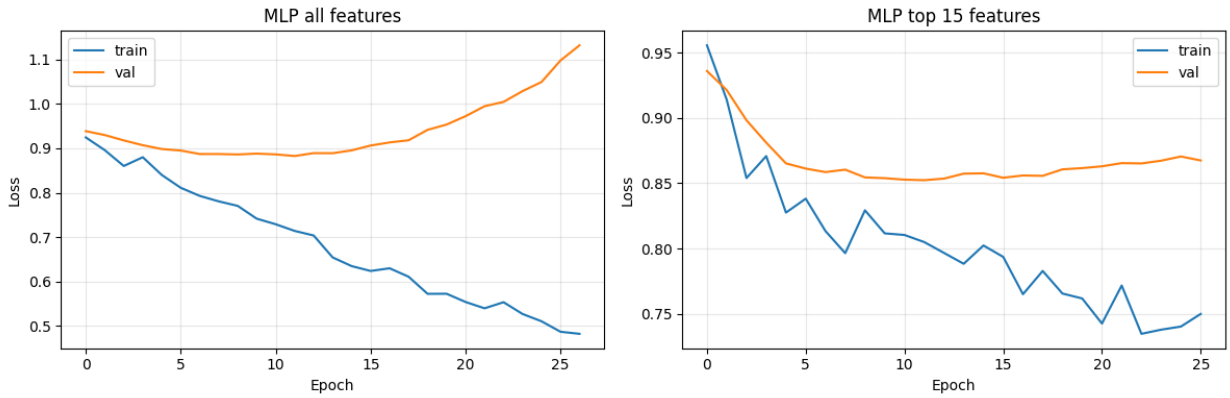

Figure 4: MLP training performance under different number of features.

4 Results & Analysis

This section evaluates and compares the performance of Gradient Boosting (GB), the Random Forest (RF) model implemented in Phase 1, and the Multilayer Perceptron (MLP) under different numbers of features.

4.1 Model Performance Comparison

Table 2 and Figure 5 illustrate the performance of different models under two feature settings, evaluated using F1 score and ROC-AUC. The results show that feature selection had a positive impact on GB and MLP, but a negative impact on RF. Meanwhile, MLP achieved the highest overall performance, maintaining the highest F1 score and ROC-AUC.

Model	Feature Set	F1 Score	ROC-AUC
Gradient Boosting	All Features	0.392	0.665
Gradient Boosting	15 Features	0.517	0.677
Random Forest	All Features	0.453	0.664
Random Forest	15 Features	0.356	0.687
Neural Network (MLP)	All Features	0.525	0.671
Neural Network (MLP)	15 Features	**0.545**	**0.704**

Table 2: Performance of models under different feature settings.

4.2 Analysis

Based on the results, the following conclusions can be derived:

- For GB, this may be because the model is relatively sensitive to noise. Removing some features reduces noise, thereby improving prediction performance.

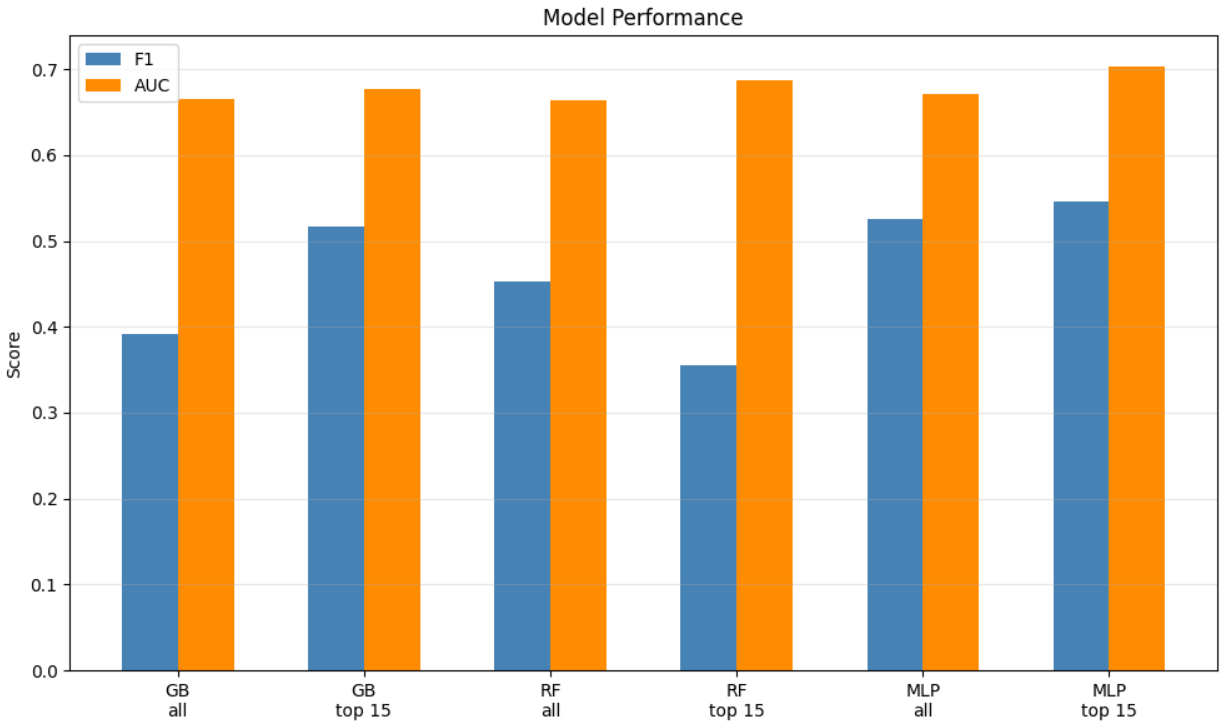

Figure 5: Model performance across different feature settings.

- For MLP, reducing the number of input features helps alleviate overfitting. During training, it can also be observed that the model performs more stably on the validation set after feature selection, thereby enhancing the model's predictive capability.

- In contrast, the F1 score for RF decreased after feature selection. This is possibly because Random Forests rely on the combination of multiple decision trees across different feature subsets to maintain model performance. When the number of features is reduced, the diversity of feature subsets decreases, thereby weakening the model's predictive capability.

In conclusion, although feature selection did not have a positive effect on all models, it did improve model performance overall. This actually exceeded my expectation, as the original purpose of using feature selection was to reduce the dimensionality of the data while maintaining predictive performance. Furthermore, while the MLP achieved the best results, this is partly because Gradient Boosting was implemented using a library, Random Forest was implemented manually, and the MLP received more thorough hyperparameter tuning. Therefore, this result does not imply that a simple MLP structure is necessarily optimal, but the outcomes achieved in this project are still satisfactory.

5 Reflection

My favorite part: My favorite part of this project was building the neural network, because it allowed me to experiment with different model architectures and parameters. It was really interesting to see the model's performance improve after adjusting its structure or parameters.

My least favorite part: My least favorite part of this project was having to adjust different models to the dataset in order to ensure a fair comparison of their performance. For example, since the Random Forest model in Phase 1 was implemented manually rather than using a library, I had to go back to the Phase 1 section and modify the code so that the Random Forest could also calculate the AUC. However, this process did give me a deeper understanding of the Random Forest model.

Most Changed Perspective: I had originally thought that the primary purpose of feature selection was to reduce the dimensionality of the data, but in practice, I found that it also helps eliminate noise caused by excessive features, which actually improves the model's performance. This has changed my perception of the method, and I now find it extremely useful. I will try this approach in similar tasks moving forward.

Future Improvements: During model training, I actually noticed that the model was overfitting. This was particularly evident when the model was trained on the full dataset. If I had more time, I would like to thoroughly investigate the causes of overfitting and explore whether there are effective methods to better address this issue.

Additional Ideas: Due to time constraints, I've only implemented a relatively basic MLP model. If I get the chance, I'd like to experiment with other models that might be suitable for different types of tasks, such as RNNs or CNNs, because I've noticed in other projects that these models sometimes perform surprisingly well on tasks where they might not seem like the best fit. Additionally, since I haven't actually built a Transformer model before, I'd also like to give that a try if the opportunity arises.

Recommendations: If the workload is manageable, I think projects that align with current AI trends would be quite interesting. Such as the currently popular agent-based systems and AI content generation. These tasks can help students better understand the current application scenarios in the AI field, which will be helpful in addressing issues they may encounter in their future careers.

Time Spent: Phase 1 took me about 8 hours, while Phase 2 took me about 16–18 hours (not including the report).

References

[1] GeeksforGeeks. Feature selection techniques in machine learning, 2025. URL: https://www.geeksforgeeks.org/machine-learning/feature-selection-techniques-in-machine-learning/.

[2] GeeksforGeeks. Variance threshold, 2025. URL: https://www.geeksforgeeks.org/machine-learning/variance-threshold/.

[3] GeeksforGeeks. Pearson correlation coefficient, 2025. URL: https://www.geeksforgeeks.org/maths/pearson-correlation-coefficient/.

[4] Kai Mei, Meifang Tan, Zhihui Yang, and Shaoyue Shi. Modeling of feature selection based on random forest algorithm and pearson correlation coefficient. *Journal of Physics: Conference Series*, 2219(1):012046, apr 2022. doi:10.1088/1742-6596/2219/1/012046.

[5] GeeksforGeeks. Binary cross-entropy (log loss) for binary classification, 2025. URL: https://www.geeksforgeeks.org/deep-learning/binary-cross-entropy-log-loss-for-binary-classification/.